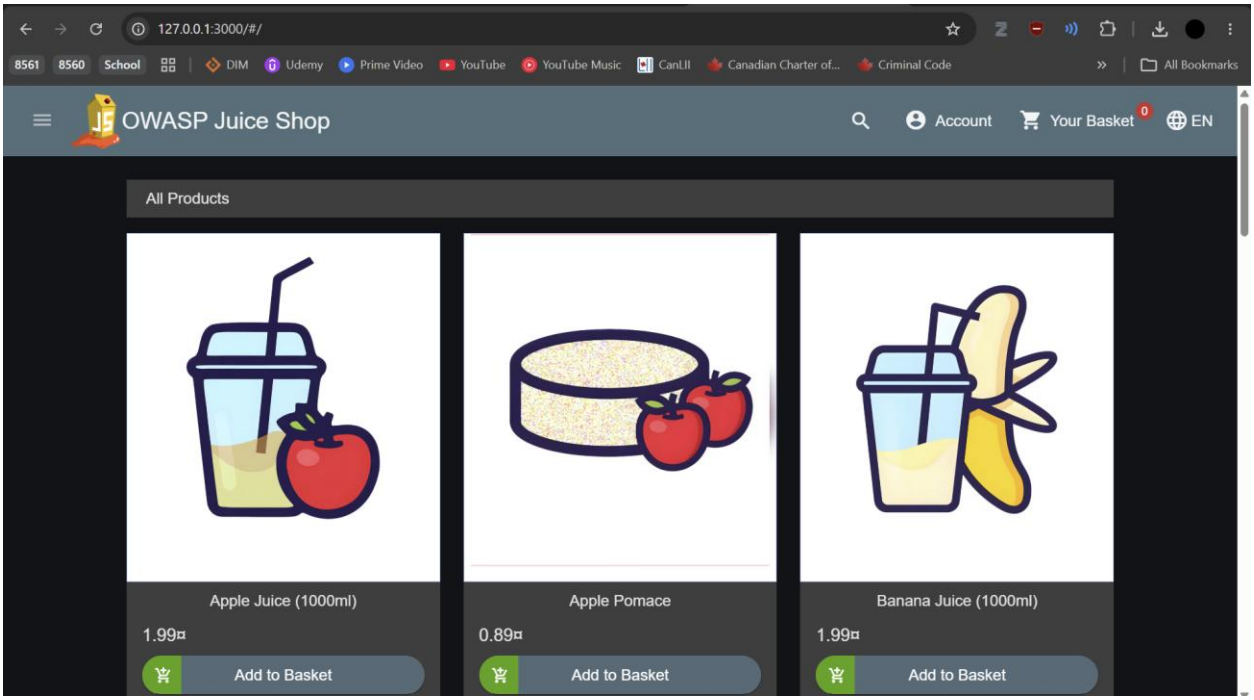
Lab 5

Lab Setup



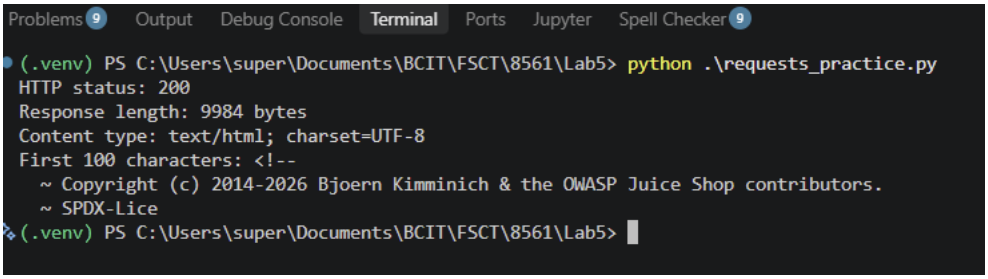
Part A - Understanding HTTP before writing a scanner

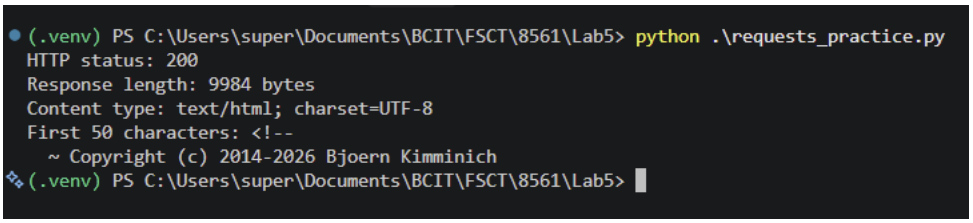
A1. Your first request



1. What is the difference between `response.status_code` and `len(response.content)`?
    a. Response.status_code returns the HTTP status number, whereas len(response.content) measures the total size of the response body.

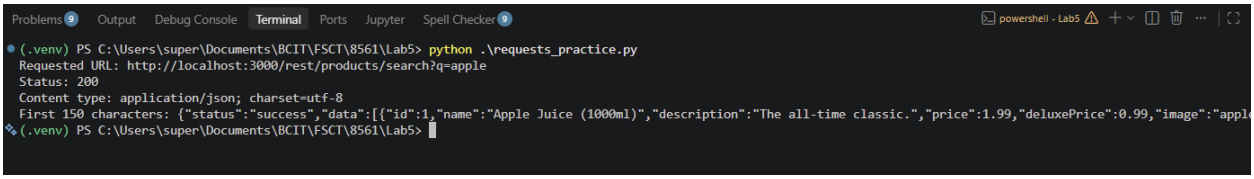
A2. Reading response headers and content



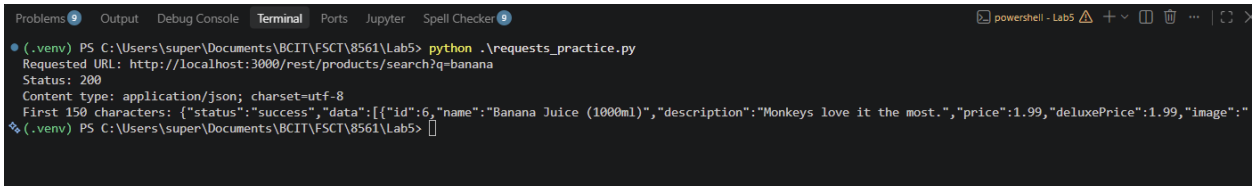


1. Explain what slicing does
    a. Slicing extracts a specific subset of characters from a string, which prevents printing out a massive HTML response body in the console.

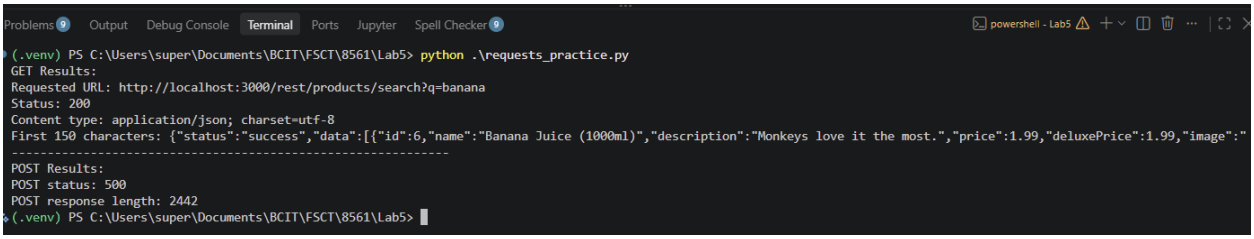
A3. Sending a search parameter



1. Run the example again using the search word `banana`. Which part of the request changed? Did the status code change?
    a. The query parameter in the URL changed from “q=apple” to “q=banana”. The status code remained as 200 because both requests were successful.



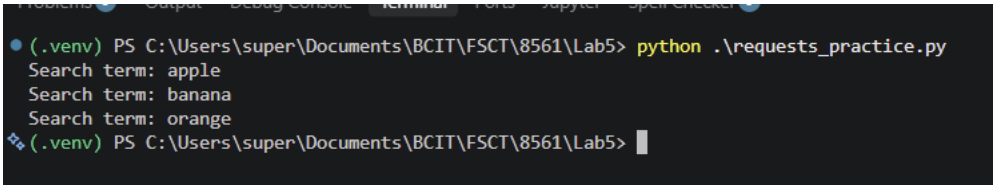
A4. GET versus POST



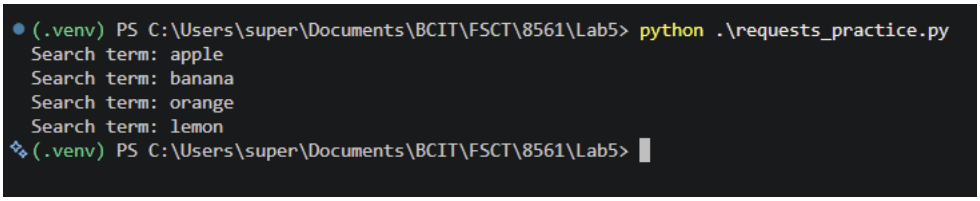
1. Compare the GET and POST results. Why does a `405` response not necessarily indicate a vulnerability?
    a. The GET request returned 200 with the product JSON, while the POST request returned status 500 because searching via a POST request is not supported by that route. A 500 or 405 error does not indicate a vulnerability on its own, it just means the server encountered an unsupported method, unhandled exception, or an error in the website's code.

Part B — Python building blocks for a scanner

B1. Lists and loops



1. Add lemon to the loop. How many times did it run?



- The loop ran 4 times

B2. Dictionaries: one record per request

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 504}
Status: 200
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

1. Add a `content_type` key to `observation` using `r.headers.get("Content-Type")`. Print that value.

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
{'method': 'GET', 'endpoint': '/rest/products/search', 'status_code': 200, 'response_length': 504, 'content_type': 'application/json; charset=utf-8'}
Status: 200
Content Type: application/json; charset=utf-8
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

B3. Combining a loop and a list of records

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
[{'input': 'apple', 'status_code': 200}, {'input': 'banana', 'status_code': 200}]
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

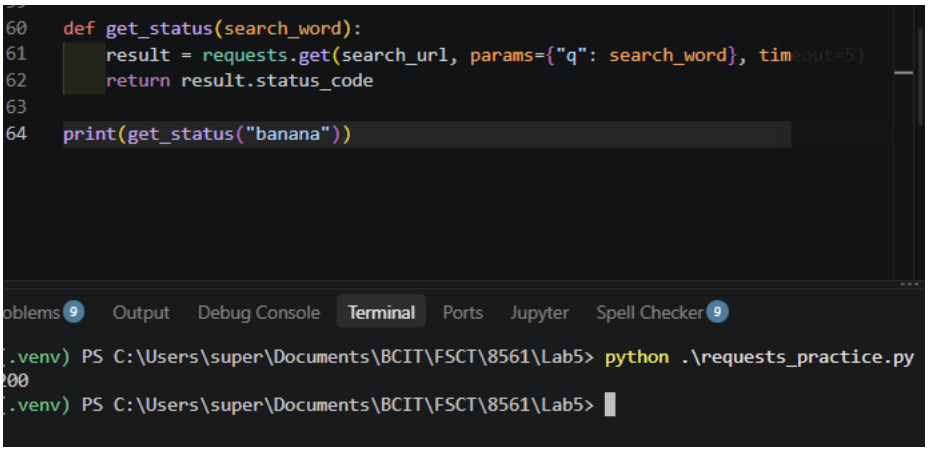
1. Add `response_length` to each dictionary. Explain why a list of dictionaries is helpful for generating a report.

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
[{'input': 'apple', 'status_code': 200, 'response_length': 1617}, {'input': 'banana', 'status_code': 200, 'response_length': 504}]
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

B4. Writing a simple function

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
200
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

1. Call `get_status("banana")`. Then explain what `search_word` means inside the function. You will decide later whether your scanner needs a function that returns a whole dictionary instead of a single number.



- The term “search_word” inside the function means it is a variable that only exists within that function. It works by passing a value through a function call then it automatically gets assigned to that variable and can be used with the logic within the function.

B5. Handling network errors

```
Problems 9  Output  Debug Console  Terminal  Ports  Jupyter  Spell Checker 9
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
Connected. Status: 200
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

1. Explain why a scanner should include a timeout and handle `requests.RequestException`
    a. A timeout prevents the scanner from hanging indefinitely if an endpoint is responses. A scanner should handle “requests.RequestException” because it can catch network level failures so the script doesn’t crash.

Part C — Recognizing potential web security indicators

C1. Reflected input and XSS

```
Problems 11  Output  Debug Console  Terminal  Ports  Jupyter  Spell Checker 11

(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
Marker was not found in this response
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

1. Test the non-executable HTML string `<u>fsct8561-test</u>` as a search term. Does it appear unchanged? If so, does that prove XSS? Explain why or why not. Keep the tests limited to the local lab instance.

```
71
72  marker = "<u>fsct8561-test</u>"
73  marker_response = requests.get(search_url, params={"q": marker}, timeout=5)
74
75  if marker in marker_response.text:
76      print("Marker appears in the response – investigate its context")
77  else:
78      print("Marker was not found in this response")
```

```
Problems 12  Output  Debug Console  Terminal  Ports  Jupyter  Spell Checker 12

(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
Marker was not found in this response
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

- The string appears unchanged in the response text. This does not prove XSS because simple reflection only shows that input is echoed back, traditional XSS requires the browser to execute it as code.

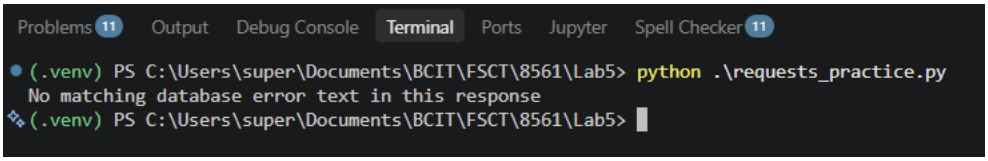
C2. SQL injection indicators

```
Problems 10  Output  Debug Console  Terminal  Ports  Jupyter  Spell Checker 10

(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
Normal input status: 200
Probe input status: 200
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

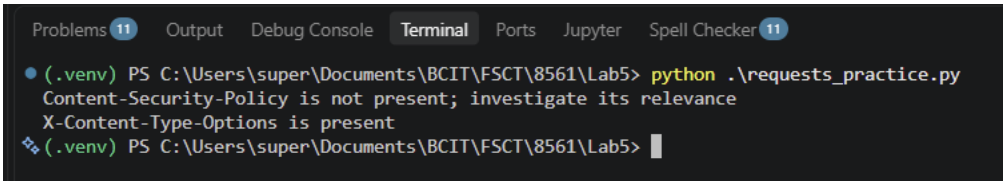
1. Explain what you observed. Why is comparing the probe response to a normal baseline more informative than looking at the probe alone?
    a. Both the normal input and the probe input returned status 200. Comparing the probe response to a normal baseline is more informative because it lets you look for changes in behavior or response length. Just looking at the probe alone does not provide context for what normal looks like.

C3. Checking for database error messages

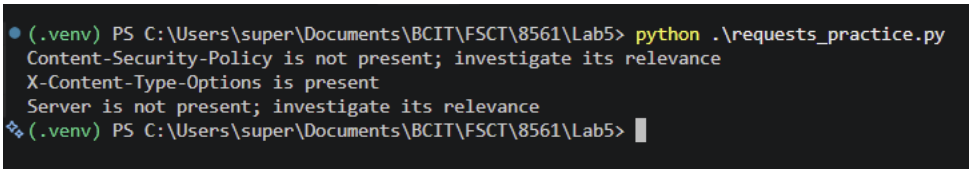


1. Suggest two additional database-error phrases that a detector might check. Explain why a matching phrase alone cannot establish successful SQL injection.
   - SQLITE_AUTH and SQLITE_CANTOPEN
   - A matching phrase alone doesn't establish successful SQL injection because there may already be some existing errors within the code.

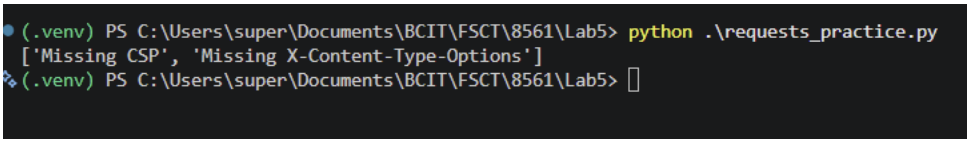
C4. Security response headers



1. Look at the Server response header using .get("Server"). What information, if any, is revealed? Why is the presence of a Server header not automatically a critical vulnerability?
   a. Using .get("Server") reveals the underlying web server software and version. The presence of a Server header is not automatically critical because it is only an informational finding, it is only beneficial for reconnaissance if an unpatched vulnerability exists for that specific version.



C5. Conditional alerts and avoiding duplicates



1. Why does the list contain only two alerts even though the loop processes three messages? In your final detector, decide what makes two alerts the "same" finding (for example, category + endpoint + observation).
   - The loop has logic where it adds items to a "set" to track if the message is already present. In python you cannot add duplicate items to a set, that is why we only see 2 alerts instead of all three.

C6. Writing observations to a CSV file

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\requests_practice.py
Saved practice_records.csv
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

Lab5.ipynb U practice_records.csv U

Lab5 > practice_records.csv

```
1  endpoint,status
2  /,200
3  /example,404
4
```

1. Open `practice_records.csv` and identify its column headings and two rows. Why is CSV useful for the final lab report?
   - It's useful because it stores the data in a clean format that can be accessed easily in code logic.

Part D - Programming Assignment

Task 1- Develop http_scanner.py

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
Connected. Status: 200
Response length: 9984 bytes

Observation 1:
HTTP Method: GET"
Endpoint: /
Status: 200"
Response Length: 9984"
Content Type: text/html; charset=UTF-8
Server Header: None



Observation 2:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None



Observation 3:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None
```

```
Observation 4:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None



Observation 5:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None



Observation 6:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None



Observation 7:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None
```

Lab5 > http_scan_results.csv

```
1  Endpoint,HTTP Method,Status,Response Length,Content Type,Server Header
2  /,GET,200,9984,text/html; charset=UTF-8,
3  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
4  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
5  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
6  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
7  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
8  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
9
```

Task 2 – Develop vulnerability_detector.py

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\vulnerability_detector.py
Category: Server Header Check
Endpoint: /
Observation: Server header is not present
Review Status: Informational


Saved vulnerability_findings.csv
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

Lab5 > vulnerability_findings.csv

```
1  Category,Endpoint,Observation,Review Status
2  Security Misconfiguration,/,Response is missing header: Content-Security-Policy,Investigate missing header relevance
3  Server Header Check,/,Server header is not present,Informational
4
```

Task 3 – Server and Configuration Analysis

1. Which response headers were present and missing, and why the context matters.
    a. The “Content-Security-Policy” header was missing, while the “X-Content-Type-Options was handled. Context matters because missing security headers only creates a risk if the application lacks other defenses.
2. Whether the server reveals potentially useful implementation information.
    a. The local Juice Shop instance did not expose a detailed “Server” banner, which highlights good configuration hygiene that prevents automated tools from easily fingerprinting the backend engine.
3. Why HTTPS/TLS cannot be meaningfully assessed solely by checking the local HTTP endpoint; describe what you would examine in an actual HTTPS deployment.
    a. TLS cannot be assessed on local HTTP because plaintext traffic is unencrypted. In a real deployment, you must examine the validity of the certificate, supported cipher suites, and HSTS headers.
4. Why a missing header or server error does not automatically mean a vulnerability is exploitable.
    a. A missing header or server error is just an indicator or exception. It does not prove an attacker can successfully exploit it without manual verification.

Task 4 – Evidence and Reproducibility

1. Your Juice Shop instance running locally.

```
PS C:\Users\super> docker run --rm -d --name fsct8561-juice -p 127.0.0.1:3000:3000 bkimminich/juice-shop
d198bd19666317b3b61053b42d4b15aad29f0f3b9378bc96554399187c493756
PS C:\Users\super>
```

```
PS C:\Users\super> docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED          STATUS          PORTS                      NAMES
d198bd196663   bkimminich/juice-shop   "/nodejs/bin/node /j…"   36 minutes ago   Up 36 minutes   127.0.0.1:3000->3000/tcp   fsct8561-juice
PS C:\Users\super>
```

2. Successful execution of **both** Python scripts.

http_scanner.py

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
Connected. Status: 200
Response length: 9984 bytes

Observation 1:
HTTP Method: GET"
Endpoint: /
Status: 200"
Response Length: 9984"
Content Type: text/html; charset=UTF-8
Server Header: None



Observation 2:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None



Observation 3:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None
```

```
Observation 4:
HTTP Method: GET"
Endpoint: /rest/products/search
Status: 200"
Response Length: 26963"
Content Type: application/json; charset=utf-8
Server Header: None



Observation 5:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None



Observation 6:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None



Observation 7:
HTTP Method: POST"
Endpoint: /rest/products/search
Status: 500"
Response Length: 2442"
Content Type: text/html; charset=utf-8
Server Header: None
```

Vulnerability_detector.py

```
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5> python .\vulnerability_detector.py
Category: Server Header Check
Endpoint: /
Observation: Server header is not present
Review Status: Informational


Saved vulnerability_findings.csv
(.venv) PS C:\Users\super\Documents\BCIT\FSCT\8561\Lab5>
```

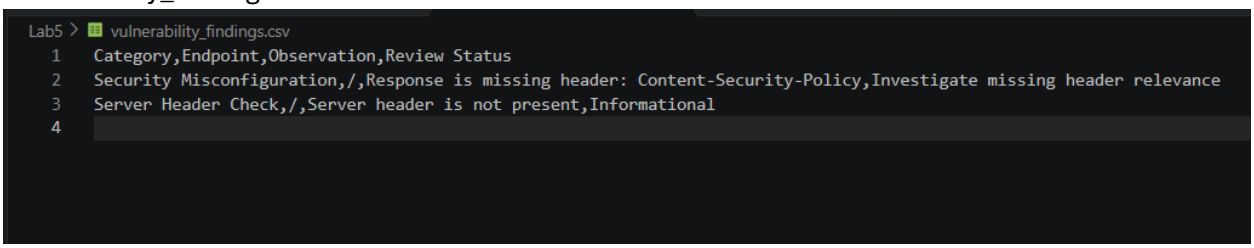
3. The generated CSV files and representative entries.

http_scan_results.csv

Lab5 > http_scan_results.csv

```
1  Endpoint,HTTP Method,Status,Response Length,Content Type,Server Header
2  /,GET,200,9984,text/html; charset=UTF-8,
3  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
4  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
5  /rest/products/search,GET,200,26963,application/json; charset=utf-8,
6  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
7  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
8  /rest/products/search,POST,500,2442,text/html; charset=utf-8,
9
```

Vulnerability_findings.csv



Task 5 – Reflection and Security Analysis

My custom http_scanner.py script successfully logged HTTP status, such as status 200 for standard GET requests, alongside response lengths and content types. Meanwhile, my vulnerability_detector.py script flagged potential security indicators, specifically highlighting missing headers like "Content-Security-Policy" and evaluating input reflection markers. It is critical to distinguish hard evidence, such as concrete HTTP status codes or missing header strings, rather than assuming a server error or missing header proves an active vulnerability exists within the system.

If I was the security lead for Juice Shop, I'd prioritize fixing how the search endpoint handles input and server errors. currently, running simple syntax probes causes a 500 status code crash, which shows weak error handling and potential injection points that need to be sanitized.

This lab extends Lab 4 by shifting from offline, passive network packet analysis into active HTTP testing. In Lab 4, we used Scapy to load PCAP files, filter IP, TCP, and UDP packets, and parse headers or timestamps. Lab 5 on the other hand, uses the Python requests library to actively query web endpoints, send query parameters, and inspect HTTP response bodies and headers.

If the application were to use HTTPS instead of plaintext HTTP, a network packet capture would only pick up encrypted TLS traffic, handshakes, and SNI data, the actual search queries and responses are hidden. In comparison, a local Python HTTP client interacting with the application allows you to directly read unencrypted response content and headers

Finally, automated findings should always be manually verified before identifying them as confirmed vulnerabilities. Automated tools rely on simple pattern matches, missing header checks, or error codes, which frequently results in false positives or flagging minor settings that cannot be utilized by an attacker.